الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

Lecture 7 Handout:

Logic Instructions

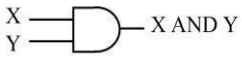
1. LOGIC INSTRUCTION AND

Syntax: `AND destination, source`

- **This instruction performs a logical AND operation on the operands and places the result in the destination.**
- **The destination operand can be either a register or memory.**
- **The source operand can be a register, memory, or an immediate value.**
- **The AND operation automatically sets the CF and OF flags to zero.**
- **The PF, ZF, and SF flags are set according to the result.**
- **The remaining flags are either unaffected or left undefined.**

Logical AND Function

Inputs		Output
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1



Example:

Let's say we want to clear certain bits in the `AX` register using the `AND` instruction.

```assembly
MOV AX, 0FFFFh  ; Load AX with hexadecimal FFFF (1111 1111 1111 1111 in binary)
AND AX, 0FF0Fh  ; Apply logical AND with 0FF0Fh (1111 1111 0000 1111 in binary)
```

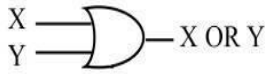
Explanation:

- Before `AND`: `AX = FFFFh` (binary: `1111 1111 1111 1111`)
- AND Mask: `0FF0Fh` (binary: `1111 1111 0000 1111`)
- After `AND`: `AX = 0FF0Fh` (binary: `1111 1111 0000 1111`)

2. LOGIC INSTRUCTION OR

Logical OR Function

Inputs		Output
X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1



Syntax: `OR destination, source`

- **The destination and source operands are ORed, and the result is placed in the destination.**
- **This can set certain bits of an operand to 1.**
- **The destination operand can be a register or memory.**
- **The source operand can be a register, memory, or an immediate value.**
- **The flags are set in the same way as for the AND instruction:**
- **CF and OF are reset to zero.**
- **SF, ZF, and PF are set according to the result.**
- **All other flags are unaffected.**

Example:

Suppose we want to set certain bits in the `AX` register using the `OR` instruction.

```assembly
MOV AX, 0F0F0h  ; Load AX with hexadecimal F0F0 (1111 0000 1111 0000 in binary)
OR AX, 0F00Fh   ; Apply logical OR with 0F00F (1111 0000 0000 1111 in binary)
```

Explanation:

- Before `OR`: `AX = F0F0h` (binary: `1111 0000 1111 0000`)
- OR Mask: `0F00Fh` (binary: `1111 0000 0000 1111`)
- After `OR`: `AX = F0FFh` (binary: `1111 0000 1111 1111`)

Example 2

Show the results of the following:

```
MOV AX,0504        ;AX = 0504
OR  AX,0DA68H      ;AX = DF6C
```

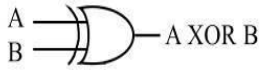
Solution:

```
0504H   0000 0101 0000 0100
DA68H   1101 1010 0110 1000
DF6C    1101 1111 0110 1100
```

3. LOGIC INSTRUCTION XOR

Logical XOR Function

Inputs		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



Syntax: `XOR destination, source`

- **The XOR instruction performs an exclusive OR operation on the operands and places the result in the destination.**
- **The result bits are set to 1 if the corresponding bits of the operands are not equal; otherwise, they are reset to 0.**
- **The flags are set in the same way as for the AND instruction:**
- **CF and OF are reset to zero.**
- **SF, ZF, and PF are set according to the result.**
- **The operand rules are the same as in the AND and OR instructions:**
- **The destination operand can be a register or memory.**
- **The source operand can be a register, memory, or an immediate value.**
- **XOR can be used to check if two registers have the same value. The instruction XOR BX, CX will set the Zero Flag (ZF) to 1 if both registers have the same value.**
- **A commonly used application of XOR is to toggle the bits of an operand.**

```
XOR  AL,04H        ;XOR  AL with 0000 0100
```

For example, toggling bit 2 of register AL will flip it to the opposite value, while all other bits remain unchanged.

Example:

Let's say we want to toggle certain bits in the `AX` register using the `XOR` instruction.

```assembly
MOV AX, 0F0F0h  ; Load AX with hexadecimal F0F0 (1111 0000 1111 0000 in binary)
XOR AX, 0FFFFh  ; Apply logical XOR with FFFF (1111 1111 1111 1111 in binary)
```

Explanation:

- Before `XOR`: `AX = F0F0h` (binary: `1111 0000 1111 0000`)
- XOR Mask: `FFFFh` (binary: `1111 1111 1111 1111`)
- After `XOR`: `AX = 0F0Fh` (binary: `0000 1111 0000 1111`)

Example

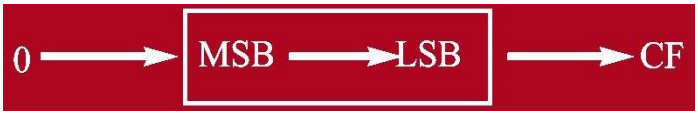
Show the results of the following:

```
MOV     DH,54H
XOR     DH,78H
```

Solution:

54H	0	1	0	1	0	1	0	0
78H	0	1	1	1	1	0	0	0
2C	0	0	1	0	1	1	0	0

4. LOGIC INSTRUCTION SHR



- **SHR - Logical Shift Right.**
- **The operand is shifted to the right, bit by bit.**
- **With each shift, the least significant bit (LSB) is moved to the carry flag (CF).**
- **The most significant bit (MSB) is filled with 0.**

Example

Show the result of SHR in the following:

```
MOV   AL,9AH
MOV   CL,3    ;set number of times to shift
SHR   AL,CL
```

Solution:

```
9AH =      10011010
           01001101      CF = 0  (shifted once)
           00100110      CF = 1  (shifted twice)
           00010011      CF = 0  (shifted three times)
```

After shifting right three times, AL = 13H and CF = 0.

- **If the operand is to be shifted only once, this is specified in the SHR instruction itself.**

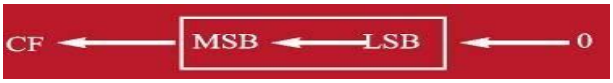
```
MOV   BX,0FFFFH    ;BX=FFFFH
SHR   BX,1         ;shift right BX once only
```

After the shift, BX = 7FFFH and CF = 1.

- **Immediate addressing mode is not allowed for the SHIFT instruction.**

```assembly
SHL AX, 3      ; This is NOT allowed in many assembly languages
```

5. LOGIC INSTRUCTION SHL



- **SHL - Logical shift left, the reverse of SHR.**
- **After every shift, the LSB is filled with 0.**
- **MSB goes to CF.**
- **All rules are the same as for SHR.**

Example :

Show the effects of SHL in the following:

```
MOV    DH,6
MOV    CL,4
SHL    DH,CL
```

Solution:

```
                    00000110
CF=0                00001100      (shifted left once)
CF=0                00011000
CF=0                00110000
CF=0                01100000      (shifted four times)
```

After the four shifts left, the DH register has 60H and CF = 0.

can also be coded as:

```
MOV    DH,6
SHL    DH,1
SHL    DH,1
SHL    DH,1
SHL    DH,1
```

6. Logic Instructions (SAR / SAL)

1. Signed Numbers:
   - `SAR` (Shift Arithmetic Right) and `SAL` (Shift Arithmetic Left) handle signed numbers differently from `SHR` (Shift Right) and `SHL` (Shift Left).
   - `SAL` is equivalent to `SHL` for shifting left.

2. SAR with Signed Numbers:
   - `SAR` correctly maintains the sign bit during shifts, making it suitable for arithmetic shifts of signed numbers.

Example:

```assembly
AL = 1110 0000   ; AL is -32.
SAR AL, 1        ; Shift AL right by 1 using SAR.
; Result: AL = 1111 0000 (which is -16 in decimal).

SAR AL, 2        ; Shift AL right by 2.
; Result: AL = 1111 1000 (which is -8 in decimal).
```

Rotate Instructions (ROR, ROL and RCR, RCL)

- **Bitwise Rotation Instructions (ROR, ROL, RCR, RCL)**

— The ***ROR*** (Rotate Right), ***ROL*** (Rotate Left), ***RCR*** (Rotate through Carry Right), and ***RCL*** (Rotate through Carry Left) instructions are designed specifically to perform bitwise rotations on an operand. These instructions allow the program to rotate an operand either to the right or left. When an operand needs to be rotated multiple times, the count is specified in the ***CL*** register.

— Operands for these rotations can reside in either a register or memory. Bitwise rotations are categorized into two types:

- **Simple Rotation (*ROR, ROL*)**: This rotation shifts bits circularly within the operand.
- **Rotation through the Carry (*RCR, RCL*)**: This rotation includes the carry flag as an additional bit, enabling rotations through the carry.

1. ROTATE INSTRUCTION ROR

Example:

```
        MOV     AL,36H          ;AL=0011 0110
        ROR     AL,1            ;AL=0001 1011   CF=0
        ROR     AL,1            ;AL=1000 1101   CF=1
        ROR     AL,1            ;AL=1100 0110   CF=1
;or:
        MOV     AL,36H          ;AL=0011 0110
        MOV     CL,3            ;CL=3 number of times to rotate
        ROR     AL,CL           ;AL=1100 0110 CF=1
;the operand can be a word:
        MOV     BX,0C7E5H       ;BX=1100 0111 1110 0101
        MOV     CL,6            ;CL=6 number of times to rotate
        ROR     BX,CL           ;BX=1001 0111 0001 1111 CF=1
```

If the operand is to be rotated once, the value 1 is specified. If the operand is to be rotated multiple times, the number of rotations is stored in the CL register.

2. ROTATE INSTRUCTION ROL

Example:

```
        MOV     BH,72H          ;BH=0111 0010
        ROL     BH,1            ;BH=1110 0100   CF=0
        ROL     BH,1            ;BH=1100 1001   CF=1
        ROL     BH,1            ;BH=1001 0011   CF=1
        ROL     BH,1            ;BH=0010 0111   CF=1
;or:
        MOV     BH,72H          ;BH=0111 0010
        MOV     CL,4            ;CL=4 number of times to rotate
        ROL     BH,CL           ;BH=0010 0111   CF=1

;The operand can be a word:
        MOV     DX,672AH        ;DX=0110 0111 0010 1010
        MOV     CL,3            ;CL=3 number of times to rotate
        ROL     DX,CL ;DX=0011 1001 0101 0011 CF=1
```

If the operand is to be rotated once, the value 1 is specified. If the operand is to be rotated multiple times, the number of rotations is stored in the CL register.

3. ROTATE INSTRUCTION RCR

Example:

```
        CLC                     ;make CF=0
        MOV     AL,26H          ;AL=0010 0110
        RCR     AL,1            ;AL=0001 0011 CF=0
        RCR     AL,1            ;AL=0000 1001 CF=1
        RCR     AL,1            ;AL=1000 0100 CF=1
or:
        CLC                     ;make CF=0
        MOV     AL,26H          ;AL=0010 0110
        MOV     CL,3            ;CL=3 number of times to rotate
        RCR     AL,CL           ;AL=1000 0100 CF=1

;the operand can be a word
        STC                     ;make CF=1
        MOV     BX,37F1H        ;BX=0011 0111 1111 0001
        MOV     CL,5            ;CL=5 number of times to rotate
        RCR     BX,CL           ;BX=0001 1001 1011 1111 CF=0
```

4. ROTATE INSTRUCTION RCL

Example:

```
        STC                     ;make CF=1
        MOV     BL,15H          ;BL=0001 0101
        RCL     BL,1            ;0010 1011 CF=0
        RCL     BL,1            ;0101 0110 CF=0
or:
        STC                     ;make CF=1
        MOV     BL,15H          ;BL=0001 0101
        MOV     CL,2            ;CL=2 number of times for rotation
        RCL     BL,CL           ;BL=0101 0110 CF=0

;the operand can be a word:
        CLC                     ;make CF=0
        MOV     AX,191CH        ;AX=0001 1001 0001 1100
        MOV     CL,5            ;CL=5 number of times to rotate
        RCL     AX,CL           ;AX=0010 0011 1000 0001 CF=1
```

If the operand is to be rotated once, the value 1 is specified. If the operand is to be rotated multiple times, the number of rotations is stored in the CL register.